

ECA1 Project 1 Group 8

Mansi Bhardwaj, Thomas Kas, Jeffrey van Velzen

Configuration	ISA Extensions	Tuning Options
base	I X	-
medium	C I M X	-
performance	C I M X	FAST_MUL

TABLE I
DIFFERENCES IN ARCHITECTURES

I. HARDWARE TESTING AND DESCRIPTION

From uploading each hardware configuration, *base*, *medium*, and *performance*, we have identified the differences as shown in Table I. Note that each configuration also has the ISA extensions *Zicsr*, *Zicntr*, *Zifencei*, *Zihpm*, and *Smpmp*, which have been left out of the table for brevity. The ISA extensions have the following meaning:

- I: Integer base ISA (always enabled)
- X: NEORV32-specific extension (always enabled)
- C: Compressed instructions
- M: Integer multiplication/division instructions
- *Zicsr*: Control and status register access instructions (always enabled)
- *Zicntr*: Base counters
- *Zifencei*: Instruction stream synchronization instruction (always enabled)
- *Zihpm*: Hardware performance monitors
- *Smpmp*: Physical memory protection

The *FAST_MUL* CPU tuning option signifies that fast multiplication is supported. This speeds up multiplication significantly, requiring only a few CPU cycles to complete. Without it, a bit-serial multiplier is used that only computes a single result bit per cycle, requiring 32 cycles per multiplication.

What they have in common is that they all run at a 100 MHz clock rate.

We test the different soft-core configurations by running the provided convolution code on each configuration, compiling them with each available ISA extension for each configuration. The results are shown in Table II.

The columns have the following meanings:

- *cycle*: active clock cycles
- *instret*: retired instructions
- *HPM03*: compressed instructions
- *HPM04*: instruction dispatch wait cycles
- *HPM05*: ALU wait cycles
- *HPM06*: branch instructions
- *HPM07*: control flow transfers
- *HPM08*: load instructions
- *HPM09*: store instructions
- *HPM10*: load/store wait cycles

- *HPM11*: entered traps

From these results, we can identify bottlenecks for each configuration. To start, *base* is the second-highest performant configuration, and has the highest number of branch instructions. We can see that it has a high number of load/store wait cycles, suggesting a high number of cache-misses. This is a large bottle-neck that could be quite easy to address by accessing memory in a more linear way. Related to this is a high number of load instructions, which we should be able to reduce, also addressing the memory bottle-neck. There is quite a high number of instruction dispatch wait cycles and branch instructions, which we might be able to address using compiler flags. Finally, we have a relatively large number of ALU wait cycles, which likely means the ALU cannot continue due to data dependencies or complex operations. This might be addressed by using less dependent arithmetic operations.

Next, *medium* needed the fewest clock cycles to finish. This suffers from the same general bottle-necks as *base*, but with some more load/store wait cycles, more load instructions, and more ALU wait cycles. However, they are all in the same ballpark (apart from the ALU wait cycles), so we should be able to address them with similar strategies.

Finally, *performance* was the worst-performing configuration by number of clock cycles, which is an interesting result given that it has the most complete instruction set out of the bunch, with specific *FAST_MUL* tuning. It also suffers from the same general bottle-necks as the other configurations, however it has the highest number of load/store wait cycles, load instructions, and instruction dispatch wait cycles. Furthermore, it has a lower amount of ALU wait cycles, suggesting that either the *FAST_MUL* tuning makes this more efficient, or that other wait cycles mean that the ALU is idle for longer between operations. Again, we can address the bottle-necks with similar strategies as for the other configurations.

II. SOFTWARE OPTIMIZATION AND DESCRIPTION

For software optimization, we applied a combination of some code changes and additional compiler flags to reduce cycle counts. Our main code optimizations are constant substitution for kernel values, pointer arithmetic to avoid repeated index calculations, and using a temporary sum value to prevent repeated storing to the output array, which is relatively expensive. On the compiler side, we tested the difference between *-O0* and *-O1*, as well as targeting loop unrolling (*-funroll-loops*), frame/vectorization control (*-fomit-frame-pointer*, *-fno-tree-vectorize*), and sub-expression elimination (*-fcse-follow-jumps*).

-O0 represents not doing any compiler optimizations, while *-O1* enables a few different compiler flags aimed at reducing execution time and code size. Specifically, it enables the following list of compiler flags:

Configuration	cycle	instret	HPM03	HPM04	HPM05	HPM06	HPM07	HPM08	HPM09	HPM10	HPM11
base	571898	134261	0	64131	23958	34426	31857	37533	2265	69407	0
medium	571892	126677	20804	64090	36630	31802	31105	39229	2265	71922	0
performance	571900	130505	20804	66642	18774	33078	32381	40505	2265	74474	0

TABLE II
SOFT-CORE CONFIGURATION TESTING RESULTS

-fdefer-pop -fdelayed-branch
 -fguess-branch-probability
 -fcprop-registers -floop-optimize
 -fif-conversion -fif-conversion2
 -ftree-ccp -ftree-dce
 -ftree-dominator-opts -ftree-dse
 -ftree-ter -ftree-lrs -ftree-sra
 -ftree-copyrename -ftree-fre
 -ftree-ch -fmerge-constants. In addition to enabling these flags, it enables some other optimizations that also impact execution time. These optimizations do not have any corresponding flags.

The results of our compiler optimizations are shown in Table III, Table IV, and Table V. The first row of each table shows the impact on cycle counts of just our code optimizations, which can be compared to the reference values in Table II. This already gave us a massive improvement in performance of about 15x. From there, enabling -O1 gave the biggest improvement. Then, enabling loop unrolling further improved our results, as did enabling sub-expression elimination scanning through jumps. We improved performance by about 77x for each configuration. Enabling compressed instructions during compilation made performance slightly worse. Leaving it out, we got the same performance for each configuration at 7447 clock cycles.

TABLE III
BASE (RV32I) COMPILER OPTIMIZATION RESULTS

Effort	CFLAGS	cycle
-O0		38086
-O1		10733
-O1	-funroll-loops	7463
-O1	-fomit-frame-pointer	10733
-O1	-fno-tree-vectorize	10733
-O1	-fcse-follow-jumps	10717
-O1	All of the above	7447

TABLE IV
MEDIUM (RV32IMC) COMPILER OPTIMIZATION RESULTS

Effort	CFLAGS	cycle
-O0		38494
-O1		10727
-O1	-funroll-loops	7463
-O1	-fomit-frame-pointer	10733
-O1	-fno-tree-vectorize	10733
-O1	-fcse-follow-jumps	10717
-O1	All of the above	7447

TABLE V
PERFORMANCE (RV32IMC) COMPILER OPTIMIZATION RESULTS

Effort	CFLAGS	cycle
-O0		37750
-O1		10733
-O1	-funroll-loops	7463
-O1	-fomit-frame-pointer	10733
-O1	-fno-tree-vectorize	10733
-O1	-fcse-follow-jumps	10717
-O1	All of the above	7447

III. PERFORMANCE EVALUATION

From the optimization results as previously discussed it is clear that our optimizations significantly reduced the cycle counts across all three hardware configurations. In fact, the best performing optimization is the same for each different configuration in our testing: enabling -O1 along with the flags -funroll-loops -fomit-frame-pointer -fno-tree-vectorize -fcse-follow-jumps, and not using compressed instructions (where available). Base performance varied across the different configurations, but after our optimizations, they converged to the same value: 7447 clock cycles.

IV. ENERGY-DELAY PRODUCT (EDP) EVALUATION

The Energy-Delay Product (EDP) is defined as:

$$EDP = P \times t^2$$

where P can be approximated by the number of LUTs ($P \approx |LUTs|$) and time can be estimated as the number of cycles divided by the clock frequency ($t \approx \frac{cycles}{frequency}$).

As such, we obtain the following EDP values for each configuration:

$$EDP_{base} \approx 2308 \times \left(\frac{7447}{100\,000\,000} \right)^2 \approx 1.28 \times 10^{-5}$$

$$EDP_{medium} \approx 4868 \times \left(\frac{7447}{100\,000\,000} \right)^2 \approx 2.70 \times 10^{-5}$$

$$EDP_{perf} \approx 6958 \times \left(\frac{7447}{100\,000\,000} \right)^2 \approx 3.86 \times 10^{-5}$$

We find that base achieves the lowest EDP due to its low number of LUTs, making it the most energy-efficient hardware/software combination.

ECA1 Project 2 – GPU-Accelerated 3×3 Convolution (Group 8)

Mansi Bhardwaj, Jeffrey van Velzen

CONTENTS

I	Introduction	1
II	Platform Specification and Selection Justification	2
II-A	Platform Choice Rationale	2
II-A1	Computational Alignment	2
II-A2	Architectural Advantages	2
II-B	Development System Specifications	2
III	Architecture and Implementation	2
III-A	Kernel Design	2
III-B	Host–Device Interaction	3
III-C	Correctness Verification	3
IV	Optimization Strategies	4
V	Performance Evaluation	4
VI	Discussion and Observations	4
VII	Conclusion and Future Improvements	4

I. INTRODUCTION

Convolution is a fundamental mathematical operation commonly used in digital signal processing, image processing, and convolutional neural networks (CNNs). The 3×3 convolution kernel is especially important in many embedded vision applications. However, when applied to large matrices—such as the 1300×1300 input matrix specified in this project—the computational demands increase significantly. This necessitates the use of efficient hardware acceleration to achieve practical performance.

This project investigates GPU acceleration using the NVIDIA CUDA platform. The choice of GPU is based on the inherent parallelism of convolution operations. Each of the 1.69 million output elements can be computed independently, making this problem ideally suited for GPU architecture. Furthermore, the regular data access patterns and high computational density align well with the GPU’s strengths in parallel processing and memory bandwidth utilization.

This report documents the implementation and optimization of the 3×3 convolution on GPU hardware. We focus on achieving high performance while maintaining numerical correctness. The following sections outline the platform specifications, architectural design, optimization strategies, performance evaluation, and an overall assessment of the GPU’s suitability for embedded convolution tasks.

II. PLATFORM SPECIFICATION AND SELECTION JUSTIFICATION

A. Platform Choice Rationale

The NVIDIA GPU platform with CUDA was selected based on several technical factors that align with the computational characteristics of the convolution problem.

1) Computational Alignment

nosep, left=0pt

- **Massive Data Parallelism:** The 1.69 million independent output elements map naturally to the GPU's thousands of concurrent threads, enabling high throughput.
- **Regular Memory Access:** Convolution exhibits predictable and structured memory access patterns, making efficient use of the GPU's high memory bandwidth.
- **High Arithmetic Intensity:** Each output requires 17 arithmetic operations (9 multiplications and 8 additions), allowing the GPU's arithmetic units to be effectively utilized.

2) Architectural Advantages

nosep, left=0pt

- **Memory Hierarchy:** Shared memory enables data reuse among neighboring threads, significantly reducing global memory access latency.
- **Thread Organization:** The CUDA 2D grid and block configuration aligns naturally with the structure of 2D image processing operations.
- **Mature Ecosystem:** The CUDA development environment provides explicit control over memory management, synchronization, and performance profiling.

B. Development System Specifications

A) Host CPU nosep, left=10pt

- **Processor:** AMD Ryzen 7 7840HS
- **System Memory:** 16.0 GB DDR5

B) GPU Hardware nosep, left=10pt

- **Model:** NVIDIA GeForce RTX 4050 Laptop GPU
- **CUDA Cores:** 2560 (Ada Lovelace Architecture)
- **Dedicated VRAM:** 6 GB GDDR6 (sufficient for 1300×1300 matrix)
- **Compute Capability:** 8.9

C) Software Environment

- **CUDA Toolkit:** Version 12.9
- **Compiler:** NVIDIA nvcc (no optimization flags, as per project requirement)
- **Programming Language:** CUDA C/C++

III. ARCHITECTURE AND IMPLEMENTATION

The implementation includes a CPU reference convolution, a global-memory GPU kernel, and an optimized shared-memory GPU kernel.

A. Kernel Design

The shared-memory kernel, `gpu_convolution_shared`, loads tiles of input data into shared memory to minimize redundant global memory accesses:

```

1 __global__ void gpu_convolution_shared(
2     const int32_t *matrix, const int32_t *kernel,
3     int32_t *output, int width, int height, int ksize) {
4
5     __shared__ int32_t tile[18][18];
6     int x = blockIdx.x * blockDim.x + threadIdx.x;
7     int y = blockIdx.y * blockDim.y + threadIdx.y;
8     int tx = threadIdx.x, ty = threadIdx.y;
9
10    if (x < width && y < height)
11        tile[ty][tx] = matrix[y * width + x];
12    __syncthreads();
13
14    if (x < width - ksize + 1 && y < height - ksize + 1) {
15        int64_t sum = 0;
16        for (int ky = 0; ky < ksize; ky++)
17            for (int kx = 0; kx < ksize; kx++)
18                sum += (int64_t)tile[ty + ky][tx + kx] *
19                    (int64_t)kernel[ky * ksize + kx];
20        output[y * (width - ksize + 1) + x] =
21            (int32_t)max(min(sum, (int64_t)INT32_MAX),
22                        (int64_t)INT32_MIN);
23    }
24 }

```

Listing 1: Shared-memory convolution kernel

Each 16×16 block processes a tile of the image with one-pixel halo borders, allowing data reuse by neighboring threads.

B. Host–Device Interaction

Pinned host memory is allocated with `cudaHostAlloc()` to improve transfer performance, and timing is measured using CUDA events.

```

1 cudaHostAlloc(&matrix, width * height * sizeof(int32_t),
2             cudaHostAllocDefault);
3 cudaMalloc(&d_matrix, width * height * sizeof(int32_t));
4 cudaMemcpy(d_matrix, matrix, size, cudaMemcpyHostToDevice);

```

Listing 2: Host memory allocation and data transfer

A 2D grid configuration of 16×16 threads per block is used:

```

1 dim3 block(16,16);
2 dim3 grid((output_w + block.x - 1)/block.x,
3          (output_h + block.y - 1)/block.y);

```

Listing 3: Grid and block configuration

C. Correctness Verification

The GPU output is compared element-wise with the CPU reference implementation to ensure exact numerical equivalence.

IV. OPTIMIZATION STRATEGIES

- **Shared Memory Tiling:** Reuses data among threads within each block, drastically reducing global memory latency.
- **Pinned Host Memory:** Uses `cudaHostAlloc()` for faster transfers between CPU and GPU.
- **Occupancy Tuning:** A 16×16 block size balances resource utilization and shared memory limits.
- **Loop Unrolling:** The inner convolution loops were simplified to increase instruction-level parallelism.
- **Event-based Timing:** CUDA events provide precise timing for transfer and kernel stages.

V. PERFORMANCE EVALUATION

TABLE I: Performance Results on NVIDIA RTX 4050 GPU

Operation	Time (ms)
Host $\rightarrow$ Device transfer	0.849
Kernel execution (shared memory)	1.050
Device $\rightarrow$ Host transfer	0.570
Total GPU time	2.837
Global-memory kernel	0.106
CPU baseline	32.083

The shared-memory kernel achieved an overall speedup of $11.3\times$ compared to the CPU version and approximately $30\times$ computational speedup when excluding transfer time.

VI. DISCUSSION AND OBSERVATIONS

The GPU achieved significant performance improvement due to massive parallelism and efficient shared memory use. However:

- PCIe transfer time accounted for 50% of total execution time.
- The 3×3 kernel limits arithmetic intensity; larger kernels would better exploit GPU throughput.
- Occupancy and power efficiency could be optimized further for embedded platforms.

Overall, the RTX 4050 demonstrated strong suitability for real-time convolution tasks.

VII. CONCLUSION AND FUTURE IMPROVEMENTS

The CUDA-based shared-memory convolution achieved over $11\times$ speedup versus CPU execution while maintaining numerical accuracy. Key improvements included shared-memory tiling and pinned memory usage.

Future work:

- Use asynchronous transfers and CUDA streams for overlap.
- Store kernel coefficients in constant memory.
- Extend to batched or larger-kernel convolutions.
- Evaluate embedded GPU platforms for energy efficiency.
- Explore mixed-precision arithmetic for higher throughput.